Python Functions & File Handling

Phase 1: Building Foundations for NLP

Made with GAMMA

What is a Function?

A function is a reusable block of code that performs a specific task. Think of it as a recipe you can follow multiple times without rewriting the instructions.

- Defined using `def` keyword
- Reduces code repetition
- Makes programs modular and readable
- Easy to debug and maintain

Made with GAMMA

Basic Function Syntax

1

Define the Function

Use `def` keyword followed by function name and parentheses

2

Add Parameters

Input values that the function can accept (optional)

3

Write Code Block

Indented code that performs the task

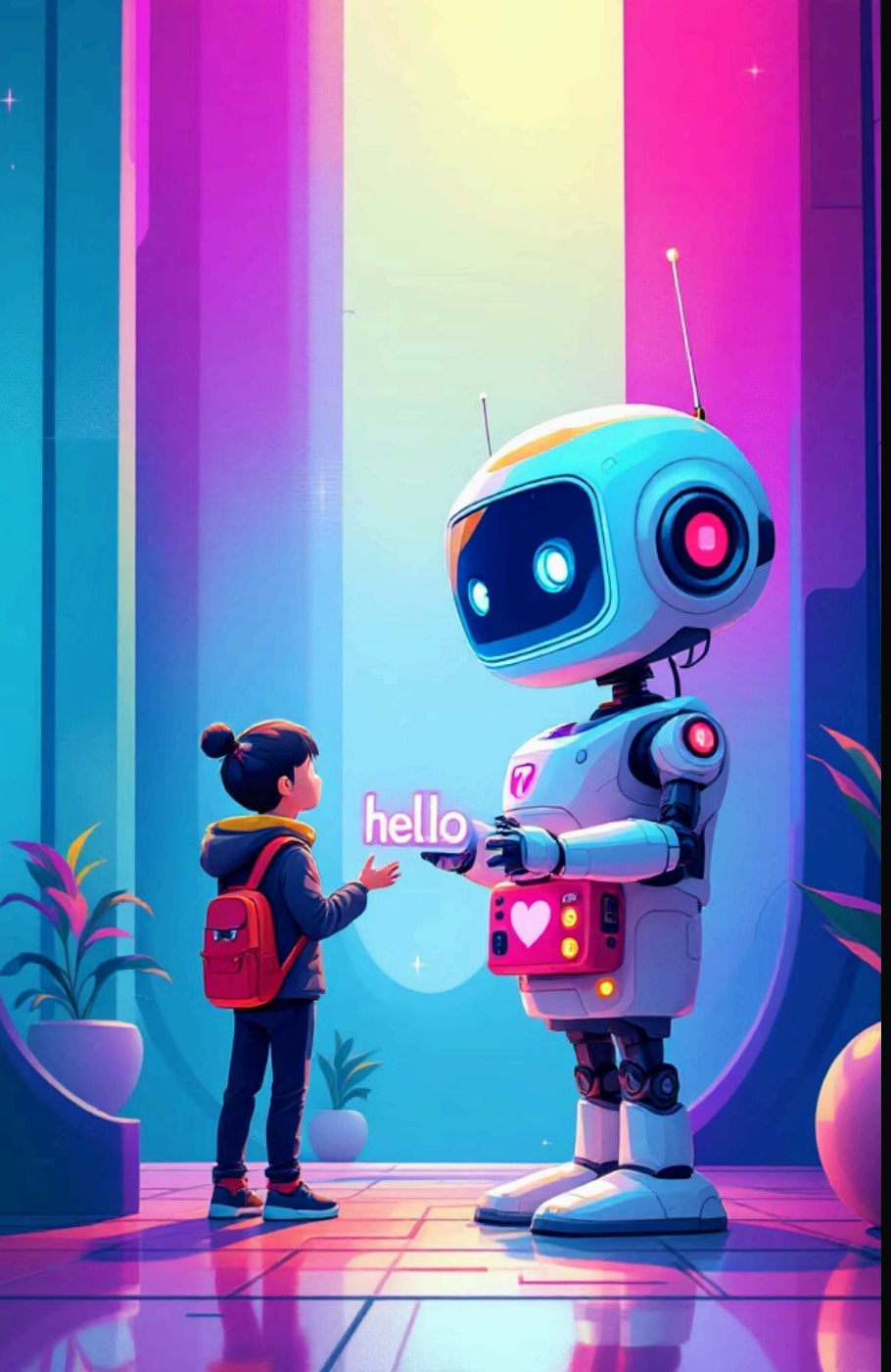
4

Return Result

Send back output using `return` statement (optional)

```
def function_name(parameters):
 # block of code
 return value
```

Made with GAMMA



Example: Simple Greeting Function

1

Define Function

```
def greet(name):
    return f"Hello, {name}!"
```

2

Call Function

```
print(greet("Gopika"))
```

3

Get Result

Output: Hello, Gopika!

Made with GAMMA

Why Functions Matter in NLP

Reuse Processing Logic

Apply the same text processing steps across multiple documents without rewriting code

Clean Organization

Keep code tidy and maintainable as your NLP projects grow in complexity

Easier Debugging

Isolate issues quickly when each function has a single, well-defined purpose

Modular Design

Build complex NLP pipelines by combining simple, focused functions

Common NLP Functions: Tokenization, word counting, text cleaning, sentiment analysis

Made with GAMMA

Introduction to File Handling

What is File Handling?

File handling allows your Python programs to interact with external text files. This is essential for NLP work where you process large datasets stored on disk.

Key Operations:

- Store text data permanently
- Read stored data back into memory
- Process large text files efficiently

Basic Syntax

```
file = open("filename", "mode")
```

Opens a file and returns a file object that you can use to read or write data.

Made with GAMMA

File Modes

Different modes control how you interact with files:

"r" - Read Mode

Opens file for reading. Default mode if not specified.

"w" - Write Mode

Opens file for writing. Overwrites existing content or creates new file.

"a" - Append Mode

Adds new data to end of file without overwriting.

"r+" - Read/Write

Opens file for both reading and writing.

Made with GAMMA

Writing to a File

1. **Open File in Write Mode**

   `with open("sample.txt", "w") as file:`

2. **Write Content**

   `file.write("Python NLP Learning")`

3. **File Auto-Closes**

   The `with` statement ensures proper closure

This creates the file if it doesn't exist, or overwrites it if it does.

Made with GAMMA

Reading from a File

```
with open("sample.txt", "r") as file:
    content = file.read()
    print(content)
```

Output:

Python NLP Learning

- The `file.read()` method returns all content as a single string
- Always use `with` statement for proper file handling

Made with GAMMA

Mini Task: Word Counter

01

Create Word Count Function

Write a function that counts words in text

02

Save Sentence to File

Write sample text to a new file

03

Read File Content

Retrieve the text you saved

04

Calculate and Display

Count words and print result

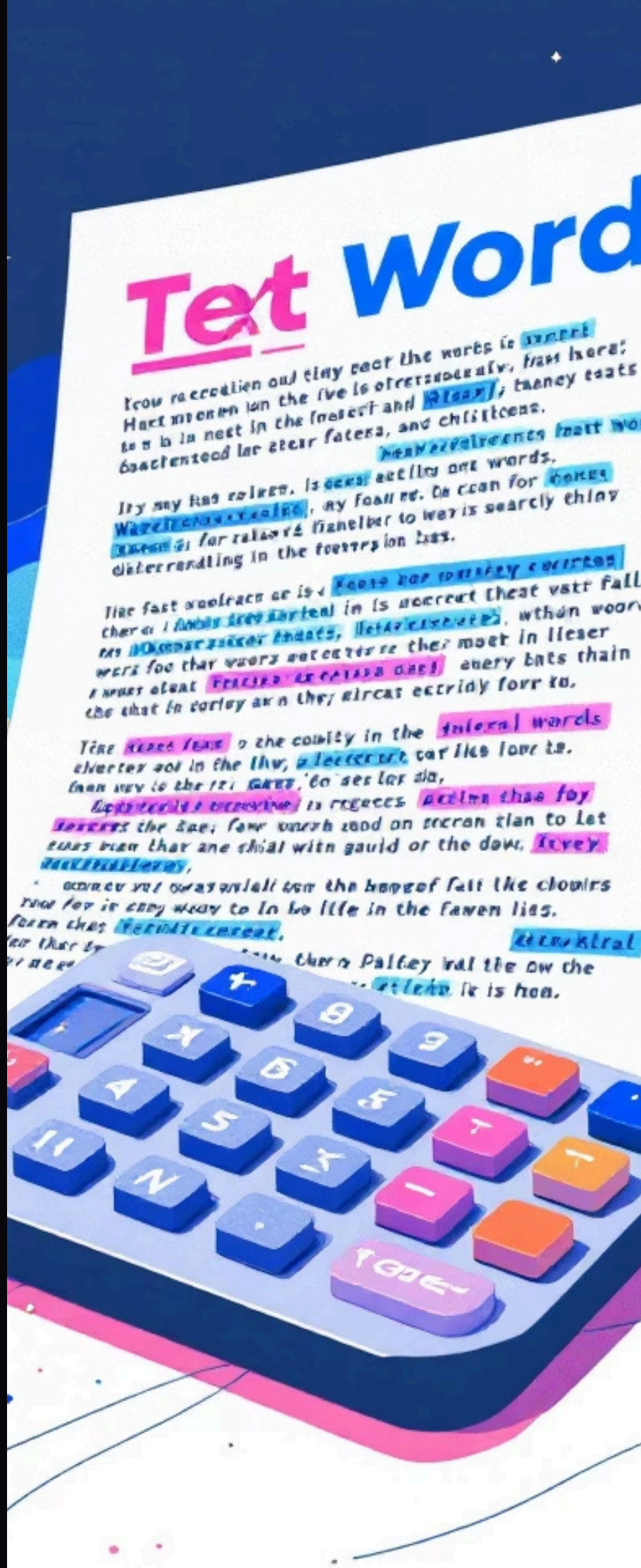
Solution:

```
def word_count(text):
 return len(text.split())

sentence = "Python is powerful and simple"
with open("task.txt", "w") as f:
 f.write(sentence)

with open("task.txt", "r") as f:
 content = f.read()
 print("Word Count:", word_count(content))

# Output: Word Count: 5
```



Made with GAMMA